UNIT 2

Optimization, Regularization and Training of Deep Neural Networks

Comprehensive Question Bank with Detailed Solutions • 17 Questions

Question 1 | Regularization in Deep Neural Networks — L1, L2 and Dropout

> *Explain the importance of regularization in deep neural networks. Compare L1 regularization, L2 regularization, and dropout.*

1. Why regularization is needed

Deep neural networks are **over-parameterised**: a modern network routinely has more trainable parameters than there are training examples. With that much capacity a network can drive its training error to almost zero by *memorising* the training set — including its noise, mislabelled samples and accidental correlations — while performing badly on unseen data. **Regularization** is any technique that constrains or penalises the model so that it prefers simpler, smoother hypotheses that generalise better.

Formally, regularization modifies the objective from pure data fitting to a trade-off between fit and complexity:

$$J(w) = L_{data}(w) + \lambda \cdot R(w)$$

where L_{data} is the training loss, $R(w)$ is a complexity penalty, and $\lambda \geq 0$ is the **regularization strength**. A large λ produces a simpler, more constrained model (risking underfitting); $\lambda = 0$ removes regularization entirely (risking overfitting). λ is a hyperparameter tuned on the validation set.

2. Importance — what regularization buys

- **Reduces the generalisation gap.** It narrows the difference between training and validation error, which is the practical definition of overfitting.
- **Controls effective capacity.** It lets you keep a large architecture — needed for optimisation reasons — while behaving like a smaller one.
- **Improves numerical conditioning.** Penalising large weights keeps activations in well-behaved ranges, reducing saturation and exploding gradients.
- **Encodes prior knowledge.** L2 encodes a preference for small, smooth weights; L1 encodes a belief that most features are irrelevant.
- **Adds robustness.** Regularised models are less sensitive to small perturbations in the input and to the particular sample of training data drawn.

3. L2 regularization (ridge / weight decay)

$$R(w) = \Sigma_i \, w_i^2 \qquad J = L_{data} + \lambda \, \Sigma \, w_i^2$$

$$\partial J/\partial w_i = \partial L_{data}/\partial w_i + 2\lambda w_i$$

$$w_i \leftarrow w_i - \eta(\partial L_{data}/\partial w_i) - 2\eta\lambda w_i$$

$$= (1 - 2\eta\lambda)\cdot w_i - \eta(\partial L_{data}/\partial w_i)$$

The factor $(1 - 2\eta\lambda) < 1$ multiplies every weight at every step, which is exactly why L2 is also called **weight decay**: weights are continuously pulled towards zero, and only a persistent data gradient can hold them away from it.

- The penalty gradient 2λw is **proportional to the weight**, so large weights are punished far more than small ones — it is a *quadratic* penalty.
- Weights shrink towards zero but essentially **never reach exactly zero**, because the pull weakens as w → 0.
- It produces **smooth, diffuse** weight vectors in which importance is spread across many inputs, which is more robust than relying on a few large weights.
- The penalty is differentiable everywhere, so it integrates cleanly with gradient descent.
- Biases are normally **excluded** from the penalty, since shrinking them serves no regularising purpose and only introduces bias in the statistical sense.

4. L1 regularization (lasso)

$R(w) = \Sigma_i |w_i|$ $\quad\quad$ $J = L_data + \lambda \Sigma |w_i|$

$\partial J/\partial w_i = \partial L_data/\partial w_i + \lambda \cdot sign(w_i)$

- The penalty gradient λ·sign(w) is a **constant magnitude** regardless of how small w is. A weight of 0.001 is pushed towards zero just as hard as a weight of 10.
- This constant pull drives many weights to **exactly zero**, producing a **sparse** model — L1 therefore performs automatic **feature selection**.
- |w| is not differentiable at w = 0; implementations use a sub-gradient (taking sign(0) = 0) or a proximal / soft-thresholding update that clips the weight to exactly zero when the step would cross it.
- Sparse models are smaller, faster at inference, and more interpretable, since the surviving non-zero weights identify the features that actually matter.
- **Elastic Net** combines both: $R(w) = \lambda_1\Sigma|w_i| + \lambda_2\Sigma w_i^2$, giving sparsity together with L2's stability when features are correlated.

5. Dropout

Dropout (Srivastava et al., 2014) is a fundamentally different idea: instead of penalising weights, it injects **noise into the architecture itself**. During each training step, every neuron in a dropout layer is independently switched off with probability p:

```
Training : r ~ Bernoulli(1 − p)   for each unit
           h̃ = (r ⊙ h) / (1 − p)      ← inverted dropout
Testing  : h̃ = h                     ← all units active, no scaling
```

The division by the keep probability (1 − p) is **inverted dropout**: it preserves the expected value of each activation, so no rescaling is needed at test time.

- **Prevents co-adaptation.** No neuron can rely on the presence of any particular other neuron, so each is forced to learn a feature that is useful on its own. This breaks up brittle, conspiratorial feature detectors.
- **Implicit ensembling.** A network with n droppable units defines 2^n possible thinned sub-networks that share weights. Training samples from this ensemble and testing with all units approximates averaging over it — an exponentially large ensemble at the cost of one model.
- **Only active during training.** At inference the full network is used deterministically.
- **Typical rates:** p = 0.5 for large fully connected layers, p = 0.2–0.3 for input or convolutional layers. Convolutional layers need less dropout because weight sharing already regularises them.
- **Cost:** training takes longer to converge (roughly 2–3×) because each step updates only a random subset of the network, and the noisy gradient makes the training loss curve visibly noisier.

6. Comparison table

Aspect	L1 Regularization	L2 Regularization	Dropout
Penalty / mechanism	$\lambda\Sigma\|w_i\|$ added to the loss	$\lambda\Sigma w_i^2$ added to the loss	Random deactivation of neurons — no loss term at all
Gradient contribution	$\lambda \cdot \text{sign}(w)$ — constant magnitude	$2\lambda w$ — proportional to the weight	None; acts through architectural noise
Effect on weights	Drives many weights to **exactly zero** (sparse)	Shrinks all weights towards zero but rarely to zero (dense, small)	Weights remain dense; robustness comes from redundancy
What it produces	Feature selection, compact interpretable models	Smooth, well-conditioned models with distributed importance	An implicit ensemble of 2^n thinned networks
Differentiability	Non-differentiable at $w = 0$ (needs sub-gradient or proximal step)	Differentiable everywhere	Not applicable
Applied at	Every training step, to weights	Every training step, to weights	Every training step, to activations
Behaviour at test time	Unchanged	Unchanged	**Disabled** — full network used
Best suited to	High-dimensional data with many irrelevant features	Almost all networks — the default choice	Large fully connected layers in deep networks
Key hyperparameter	λ	λ (weight decay)	p (drop probability)

7. Practical guidance

- **L2 (weight decay) is the default** and is used in almost every modern network, typically with λ between 10^{-5} and 10^{-3}.
- **Use L1 when you specifically want sparsity** — model compression, feature selection, or interpretability. It is far less common as a general-purpose regulariser in deep learning.
- **Dropout is complementary, not an alternative** — it is routinely combined with L2. In modern architectures with batch normalisation, dropout is used more sparingly, since batch norm already contributes noise-based regularisation.
- **These are not the only options.** Early stopping, data augmentation, batch normalisation and simply gathering more data are equally important, and augmentation is often the single most effective remedy (see Question 15).
- **Note on AdamW.** With adaptive optimisers, adding λw^2 to the loss does *not* behave like true weight decay, because the adaptive denominator rescales the penalty. AdamW decouples weight decay from the gradient, applying it directly to the parameters, and is the correct choice for Adam-trained models.

Question 2 | Numerical — Three Iterations of Gradient Descent on $J(w) = (w - 3)^2$

> *Consider the objective function $J(w) = (w - 3)^2$, with derivative $dJ/dw = 2(w - 3)$. Starting with $w_0 = 8$, perform three iterations of gradient descent using $\eta = 0.1$. Evaluate w_1, w_2 and w_3. Also calculate $J(w)$ after the third iteration.*

Given data

```
Objective  : J(w) = (w − 3)²
Gradient   : dJ/dw = 2(w − 3)
Start      : w₀ = 8
Learning rate : η = 0.1
Update rule   : w_{t+1} = w_t − η · (dJ/dw)|_{w_t}
```

The minimum of this function is clearly at $w^* = 3$, where $J = 0$. Gradient descent should therefore move w from 8 towards 3.

Iteration 1 — computing w_1

```
Gradient at w₀ = 8:
  dJ/dw = 2(8 − 3) = 2(5) = 10
w₁ = w₀ − η·(dJ/dw)
   = 8 − (0.1)(10)
   = 8 − 1
```

$w_1 = 7$

Iteration 2 — computing w_2

```
Gradient at w₁ = 7:
  dJ/dw = 2(7 − 3) = 2(4) = 8
w₂ = 7 − (0.1)(8)
   = 7 − 0.8
```

$w_2 = 6.2$

Iteration 3 — computing w_3

```
Gradient at w₂ = 6.2:
  dJ/dw = 2(6.2 − 3) = 2(3.2) = 6.4
w₃ = 6.2 − (0.1)(6.4)
   = 6.2 − 0.64
```

$w_3 = 5.56$

Value of J after the third iteration

```
J(w₃) = (w₃ − 3)²
      = (5.56 − 3)²
      = (2.56)²
```

$J(w_3) = 6.5536$

Complete iteration table

Iteration t	w_t	Gradient $2(w_t - 3)$	Step $\eta \cdot g$	$w_{(t+1)}$	$J(w_{(t+1)})$
0 (start)	8.00	10.00	1.00	**7.00**	16.0000
1	7.00	8.00	0.80	**6.20**	10.2400

Iteration t	w_t	Gradient $2(w_t - 3)$	Step $\eta \cdot g$	$w_{(t+1)}$	$J(w_{(t+1)})$
2	6.20	6.40	0.64	**5.56**	**6.5536**

Observations

- **The loss decreases monotonically:** $J(w_o)$ = 25 → 16 → 10.24 → 6.5536. Gradient descent is behaving correctly.
- **The steps shrink at each iteration** (1.00 → 0.80 → 0.64). This is characteristic of a quadratic objective: as w approaches the minimum the gradient shrinks in proportion, so the step size shrinks automatically even though η is fixed.
- **Geometric convergence.** For this problem the recurrence is w_{t+1} − 3 = (1 − 2η)(w_t − 3) = 0.8(w_t − 3), so the distance from the optimum is multiplied by 0.8 each step. Verification: 5 → 4 → 3.2 → 2.56 ✓. Convergence is exponential but never finite — w approaches 3 asymptotically.
- **Stability condition.** Convergence requires $|1 - 2\eta| < 1$, i.e. $0 < \eta < 1$. With η = 0.1 we are comfortably inside this range. Note that η = 0.5 would give $|1 - 2\eta| = 0$, converging in a *single* step, while η = 1.0 would oscillate forever between 8 and −2 and η > 1 would diverge.
- **After only three steps w is still 2.56 away from the optimum**, illustrating that η = 0.1 is quite conservative for this problem — many more iterations would be needed for a tight solution.

Question 3 | Numerical — Four Iterations of Gradient Descent on $J(w) = \frac{1}{2}(w - 6)^2$

> *Consider $J(w) = \frac{1}{2}(w - 6)^2$ with gradient $dJ/dw = w - 6$. Starting with $w_0 = 2$, perform four iterations of gradient descent using $\eta = 0.25$. Evaluate w_1, w_2, w_3 and w_4. Also calculate $J(w_4)$.*

Given data

```
Objective  : J(w) = ½(w − 6)²
Gradient   : dJ/dw = (w − 6)
Start      : w₀ = 2
Learning rate : η = 0.25
```

The minimum is at w *= 6 with J = 0. Since $w_0 = 2$ lies below the minimum, the gradient will be negative and the updates will move w upwards** — the opposite direction to Question 2, which is a useful contrast.

Iteration 1 — computing w_1

```
Gradient at w₀ = 2:  dJ/dw = 2 − 6 = −4
w₁ = 2 − (0.25)(−4)
   = 2 + 1
```

$w_1 = 3$

Iteration 2 — computing w_2

```
Gradient at w₁ = 3:  dJ/dw = 3 − 6 = −3
w₂ = 3 − (0.25)(−3)
   = 3 + 0.75
```

$w_2 = 3.75$

Iteration 3 — computing w_3

```
Gradient at w₂ = 3.75:  dJ/dw = 3.75 − 6 = −2.25
w₃ = 3.75 − (0.25)(−2.25)
   = 3.75 + 0.5625
```

$w_3 = 4.3125$

Iteration 4 — computing w_4

```
Gradient at w₃ = 4.3125:  dJ/dw = 4.3125 − 6 = −1.6875
w₄ = 4.3125 − (0.25)(−1.6875)
   = 4.3125 + 0.421875
```

$w_4 = 4.734375$

Value of $J(w_4)$

```
J(w₄) = ½(w₄ − 6)²
      = ½(4.734375 − 6)²
      = ½(−1.265625)²
      = ½(1.6018066)
```

$J(w_4) = 0.8009033$ (≈ 0.8009)

Complete iteration table

Iteration t	w_t	Gradient ($w_t - 6$)	Step $\eta \cdot g$	$w_{(t+1)}$	$J(w_{(t+1)})$
0 (start)	2.000000	−4.0000	−1.000000	**3.000000**	4.500000

Iteration t	w_t	Gradient ($w_t - 6$)	Step $\eta \cdot g$	$w_{(t+1)}$	$J(w_{(t+1)})$
1	3.000000	−3.0000	−0.750000	**3.750000**	2.531250
2	3.750000	−2.2500	−0.562500	**4.312500**	1.423828
3	4.312500	−1.6875	−0.421875	**4.734375**	**0.800903**

Observations

- **The sign of the gradient sets the direction.** Every gradient here is negative, so $-\eta \cdot g$ is positive and w increases at each step, correctly climbing towards $w^* = 6$. Compare Question 2, where the start was above the optimum and w decreased.
- **The loss falls monotonically:** $J(w_o) = 8 \rightarrow 4.5 \rightarrow 2.53 \rightarrow 1.42 \rightarrow 0.80$. The loss has been reduced by 90% in four steps.
- **Contraction factor.** The recurrence is w_{t+1} − 6 = (1 − η)(w_t − 6) = 0.75(w_t − 6). Verification of the distances from the optimum: $-4 \rightarrow -3 \rightarrow -2.25 \rightarrow -1.6875 \rightarrow -1.265625$, each 0.75 times the previous ✓.
- **Note the effect of the ½ in the objective.** Because $J = \frac{1}{2}(w - 6)^2$, the gradient is (w − 6) rather than 2(w − 6), so the contraction factor is (1 − η) instead of (1 − 2η). The ½ is included precisely to make the derivative clean, which is why the same convention appears in the squared-error loss used throughout neural network training.
- **Convergence condition:** $|1 - \eta| < 1$ requires $0 < \eta < 2$. Here $\eta = 0.25$ is safely inside the range; $\eta = 1$ would jump exactly to the optimum in one step, and $\eta \geq 2$ would fail to converge.

Question 4 | SGD, RMSProp and Adam — Working Principles and the Case for Adaptive Optimizers

> *Explain the working principles of Stochastic Gradient Descent, RMSProp, and Adam. Along with explain why adaptive optimizers are useful in training deep neural networks.*

1. Stochastic Gradient Descent (SGD)

Full-batch gradient descent computes the gradient over the entire training set before making a single update, which is prohibitively slow for large datasets. **Stochastic Gradient Descent** instead estimates the gradient from a single randomly chosen sample (or, in practice, a small mini-batch) and updates immediately:

$$g_t = \nabla_w L(w_{t-1}; x^{(i)}, y^{(i)})$$
$$w_t = w_{t-1} - \eta \cdot g_t$$

- **Unbiased but noisy.** The mini-batch gradient is an unbiased estimate of the true gradient, but it has high variance. The optimisation path is jagged rather than smooth.
- **The noise is a feature, not just a defect.** It lets the optimiser escape shallow local minima and, more importantly, saddle points, and it is believed to bias training towards flat minima that generalise better.
- **Speed.** With N = 1,000,000 samples and a batch of 128, SGD makes about 7,800 updates per epoch, whereas full-batch descent makes one.
- **SGD with momentum** is the standard refinement: $v \leftarrow \beta v + \eta \cdot g$, then $w \leftarrow w - v$. The velocity term accumulates consistent gradient directions and cancels oscillations, typically with $\beta = 0.9$.
- **Weakness:** a single global learning rate is applied to every parameter. In a network where some parameters have gradients of order 1 and others of order 10^{-4}, no single η can suit both.

2. RMSProp (Root Mean Square Propagation)

RMSProp, proposed by Geoffrey Hinton, addresses exactly that weakness by giving **each parameter its own effective learning rate**, scaled by the recent magnitude of that parameter's gradients:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2 \qquad (\beta \approx 0.9)$$
$$w_t = w_{t-1} - \eta \cdot g_t \,/\, (\sqrt{v_t} + \varepsilon) \qquad (\varepsilon \approx 10^{-8})$$

- v_t is an **exponentially weighted moving average of squared gradients** — a running estimate of the typical squared magnitude of that parameter's gradient.
- Dividing by $\sqrt{v_t}$ **normalises the step**: a parameter with persistently large gradients gets a smaller effective step, and a parameter with tiny gradients gets a proportionally larger one. The result is that all parameters progress at a comparable rate.
- The **decay factor β makes the average forgetful**, so it tracks the *current* regime rather than the whole history. This is the key improvement over AdaGrad, whose non-decaying sum of squared gradients grows without bound and eventually kills the learning rate entirely.
- **ε** is a small constant added purely to prevent division by zero when v_t is very small.
- RMSProp is particularly effective on **non-stationary objectives** and on recurrent networks.

3. Adam (Adaptive Moment Estimation)

Adam (Kingma & Ba, 2015) combines momentum's use of the **first moment** with RMSProp's use of the **second moment**, and adds a bias correction:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \qquad (\beta_1 = 0.9)$$
$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \qquad (\beta_2 = 0.999)$$

$$\hat{m}_t = m_t / (1 - \beta_1^t) \qquad \leftarrow \text{bias correction}$$
$$\hat{v}_t = v_t / (1 - \beta_2^t)$$

$$w_t = w_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

- **m_t (first moment)** is the momentum term — an exponentially weighted average of the gradients themselves, providing direction and smoothing.
- **v_t (second moment)** is the RMSProp term — an average of squared gradients, providing per-parameter step scaling.
- **Bias correction is essential.** Both m and v are initialised at 0, so early estimates are biased towards zero. Dividing by $(1 - \beta^t)$ corrects this. At t = 1 with $\beta_1 = 0.9$, the correction is $1/(1 - 0.9) = 10$ — without it the first update would be ten times too small, and for v the factor is 1000. See Question 11 for a worked demonstration.
- **Defaults that work almost everywhere:** $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.
- **AdamW** is the modern variant, in which weight decay is decoupled from the adaptive scaling and applied directly to the parameters; it is now the standard for Transformers and large models.

4. Why adaptive optimizers help in deep networks

Problem in deep networks	How adaptive methods address it
Gradients differ enormously in scale across layers. Early layers often receive gradients orders of magnitude smaller than late layers (the vanishing-gradient effect).	Dividing by $\sqrt{v}$ equalises the effective step size, so early layers are not left behind while late layers train.
No single learning rate suits all parameters. With plain SGD, an η large enough for slow parameters makes fast ones diverge.	Each parameter gets its own effective rate $\eta/\sqrt{v_t}$, derived automatically from its own gradient history.
Sparse gradients. In embeddings and NLP models, most parameters receive a gradient only rarely.	Rarely updated parameters have small accumulated v, so when a gradient finally arrives the step is large — they are not starved.
Ill-conditioned loss surfaces (ravines). Curvature varies sharply between directions, causing plain SGD to zig-zag.	The second-moment scaling acts as a cheap diagonal approximation to curvature, damping steep directions and accelerating flat ones.
Saddle points and plateaus. Gradients are near zero and plain SGD stalls.	Momentum carries the optimiser through, while the small v in flat regions inflates the step size.
Costly hyperparameter tuning. SGD's performance is highly sensitive to η and its schedule.	Adam works well with its default $\alpha = 0.001$ across a wide range of problems, dramatically reducing tuning effort.

5. A balanced view

Adaptive optimizers are not universally superior. Well-tuned **SGD with momentum** often achieves marginally better final test accuracy on large vision benchmarks, and there is evidence that Adam can converge to sharper minima that generalise slightly less well. Adam also requires storing two extra values per parameter, tripling optimiser memory. The pragmatic position adopted in practice is: **use Adam (or AdamW) as the default**, especially for Transformers, RNNs, GANs and any problem with sparse or badly scaled gradients; consider

tuned SGD with momentum plus a good schedule when squeezing out the last fraction of a percent on a well-understood vision task.

Question 5 | The Role of the Learning Rate in Gradient-Based Optimization

> *Explain the role of learning rate in gradient-based optimization. Discuss consequences of selecting a learning rate that is too small or too large.*

1. What the learning rate does

The **learning rate** η is the scalar that converts the gradient — which supplies only a *direction* and a relative magnitude — into an actual **step size** in parameter space:

```
w_{t+1} = w_t − η · ∇J(w_t)
```

The gradient tells the optimiser *which way* is downhill; η decides *how far* to travel in that direction before recomputing. It is almost universally regarded as the **single most important hyperparameter** in deep learning: a network with a well-chosen learning rate and a mediocre architecture will usually outperform an excellent architecture with a badly chosen learning rate.

Conceptually, η controls the trade-off between **speed** and **stability**. Gradient descent is built on a local linear approximation of the loss, which is only valid in a small neighbourhood. η determines how far we are willing to trust that approximation before re-evaluating.

2. Visual comparison

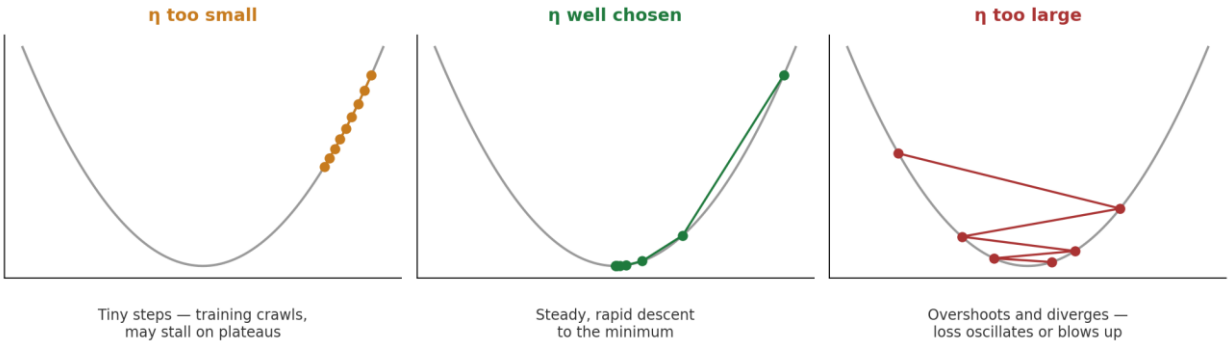


Figure 5.1 — The same quadratic objective optimised with three different learning rates.

3. Consequences of a learning rate that is too small

- **Extremely slow convergence.** Each step barely moves the parameters, so reaching a good solution may require orders of magnitude more epochs — often more compute than is available.
- **Training appears to stall.** The loss curve becomes almost flat and it is difficult to distinguish 'converged' from 'crawling', which wastes experimental time.
- **Trapped in poor solutions.** Without enough step size to escape, the optimiser settles into the nearest shallow local minimum, saddle point or plateau instead of continuing to a better region.
- **Vanishing progress in early layers.** Combined with already small backpropagated gradients, a tiny η can make the update for early-layer weights fall below floating-point resolution, so those layers effectively never train.
- **Overfitting risk through prolonged training.** Compensating for a tiny η by training for a very large number of epochs increases exposure to the training set and can lead to memorisation.

4. Consequences of a learning rate that is too large

- **Overshooting the minimum.** The step jumps past the valley floor to a point on the far wall that may be *higher* than where it started, so the loss increases rather than decreases.

- **Oscillation and divergence.** In a quadratic bowl with curvature c, updates diverge whenever $\eta > 2/c$. The loss then oscillates with growing amplitude and eventually becomes **NaN** or **Inf** — the classic sign of an excessive learning rate.
- **Numerical instability.** Large weight values push activations into saturation (sigmoid/tanh) or produce exploding activations (ReLU), and can trigger overflow in the exponential of softmax.
- **Dead ReLU units.** A single huge update can drive a unit's pre-activation permanently negative, after which its gradient is zero and it can never recover — permanently destroying capacity.
- **Skipping over good minima.** Even when it does not diverge, an overly large η bounces around the landscape and can never settle into a narrow, high-quality minimum.

5. Summary table

Learning rate	Loss-curve signature	Outcome
Far too small	Almost flat; decreases by a negligible amount per epoch	Training is impractically slow; may appear converged when it is not
Slightly too small	Smooth and steady, but takes very many epochs	Works, but wastes compute; likely to stop short of the best solution
Well chosen	Falls quickly at first, then flattens smoothly to a low value	Fast, stable convergence to a good minimum
Slightly too large	Falls quickly then plateaus at a poor value, with visible noise	Bounces around the minimum without settling; final accuracy suffers
Far too large	Increases, oscillates wildly, or becomes NaN within a few iterations	Complete divergence — training fails outright

6. Choosing and adapting the learning rate

- **Typical starting values:** $\eta = 0.001$ for Adam, $\eta = 0.01$–0.1 for SGD with momentum. These differ because Adam's adaptive denominator already normalises the step magnitude.
- **Learning-rate range test.** Train for a few hundred iterations while increasing η exponentially and plot loss against η. Choose a value about an order of magnitude below the point at which the loss starts to rise sharply.
- **Use a schedule.** No single η is optimal throughout training: a large rate early on explores quickly, while a small rate later refines the solution. Step decay, exponential decay and cosine annealing are all standard (see Question 13).
- **Warm-up.** Very deep networks and Transformers begin with a small η that ramps up over the first few epochs, because early gradients are large and unreliable.
- **Gradient clipping** is a safety net that rescales the gradient when its norm exceeds a threshold, preventing a single anomalous batch from destroying the model.
- **Adaptive optimisers reduce but do not remove the problem.** Adam still has a base learning rate α that must be set sensibly — it merely makes the choice less critical.

Question 6 | Batch Normalization and Dropout

> *Explain batch normalization and dropout. Discuss how these techniques contribute to stable training and improved generalization.*

PART A — BATCH NORMALIZATION

1. The problem it solves

As training proceeds, the weights of every layer change, so the **distribution of inputs to each subsequent layer keeps shifting**. Ioffe and Szegedy (2015) named this **internal covariate shift**. Each layer is therefore continually chasing a moving target: it must re-adapt to a new input distribution at every step, which slows training, forces small learning rates, and pushes activations into saturated regions where gradients vanish.

2. The algorithm

Batch normalization is inserted between the linear transformation and the activation function. For a mini-batch B = {x_1 … x_m} of pre-activations for one unit:

$$\mu_B = \frac{1}{m}\sum_i x_i \qquad \leftarrow \text{batch mean}$$

$$\sigma^2_B = \frac{1}{m}\sum_i (x_i - \mu_B)^2 \qquad \leftarrow \text{batch variance}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma^2_B + \varepsilon}} \qquad \leftarrow \text{normalise to mean 0, variance 1}$$

$$y_i = \gamma \cdot \hat{x}_i + \beta \qquad \leftarrow \text{scale and shift (learned)}$$

The parameters **γ and β are trainable**, one pair per unit. They are essential: forcing every layer's output to have exactly zero mean and unit variance would restrict what the network can represent — for a sigmoid it would confine activations to the near-linear central region. By learning γ and β, the network can recover any distribution it needs, *including* the identity transformation (γ = √(σ²+ε), β = μ). Normalisation is therefore a *reparameterisation* that makes optimisation easier, not a constraint on expressiveness.

Training versus inference. During training, μ and σ² come from the current mini-batch. At inference the batch may be a single sample, so BN instead uses **running (moving) averages** of the mean and variance accumulated during training. Forgetting to switch to evaluation mode is one of the most common practical bugs in deep learning.

3. Benefits of batch normalization

- **Stabilises and accelerates training.** Layer inputs keep a consistent distribution, so each layer learns against a stable target. Networks typically train several times faster in wall-clock terms.
- **Permits much higher learning rates.** Normalised activations cannot explode layer by layer, so η can often be increased by an order of magnitude — a large part of the observed speed-up.
- **Mitigates vanishing gradients.** By keeping pre-activations near zero, BN holds sigmoid and tanh units in their high-derivative region and keeps ReLU units in their active region.
- **Reduces sensitivity to weight initialisation.** Because the scale of the activations is corrected anyway, a poor initialisation is far less damaging.
- **Provides mild regularization.** The batch statistics differ from batch to batch, so each sample's normalised value carries a small amount of noise. This has an effect similar to a weak dropout and often improves generalisation on its own.
- **Smoothes the loss landscape.** Later analysis (Santurkar et al., 2018) argued that BN's real benefit is a smoother, better-conditioned optimisation surface rather than the reduction of covariate shift originally proposed.

Limitations: BN depends on the batch statistics, so it degrades badly for very small batch sizes (below about 8), it is awkward in recurrent networks where sequence lengths vary, and it introduces a train/test discrepancy. **Layer Normalization** — which normalises across features within a single sample rather than across the batch — is used instead in RNNs and Transformers for exactly these reasons.

PART B — DROPOUT

4. The algorithm

Dropout randomly deactivates neurons during training. With drop probability p (keep probability q = 1 − p), using the standard **inverted dropout** formulation:

$$\text{Training}: r_j \sim \text{Bernoulli}(q) \quad \text{independently for each unit } j$$
$$\tilde{h}_j = (r_j \cdot h_j) / q$$
$$\text{Testing}: \tilde{h}_j = h_j \qquad (\text{all units active, no scaling})$$

The division by q keeps the expected activation unchanged: $E[\tilde{h}_j] = q\cdot(h_j/q) + (1 - q)\cdot 0 = h_j$. Because the correction is applied at training time, inference is a plain deterministic forward pass — which is why this form is universally preferred.

5. Why dropout improves generalization

- **It prevents co-adaptation.** Without dropout, neurons form fragile partnerships in which one unit's error is compensated by another. Since any partner may vanish at any step, each unit must learn a feature that is independently useful.
- **It approximates an exponential ensemble.** A layer with n droppable units defines 2^n possible thinned sub-networks that share parameters. Training samples from this family, and testing with the full network approximates averaging their predictions — ensembling is one of the most reliable ways to reduce variance, and dropout provides it at the cost of a single model.
- **It adds noise, which flattens the minima found.** Solutions that survive random unit removal are necessarily robust and redundant, and such flat solutions generalise better.
- **It is a strong constraint on effective capacity** without reducing the architecture's size, so the model retains the optimisation advantages of being large.

6. Comparison of the two techniques

Aspect	Batch Normalization	Dropout
Primary purpose	Stabilise and accelerate optimisation	Improve generalisation / prevent overfitting
Mechanism	Normalises pre-activations using batch statistics, then rescales with learned γ, β	Randomly zeroes activations with probability p and rescales by 1/(1 − p)
Adds parameters?	Yes — γ and β per unit (plus running statistics)	No trainable parameters
Depends on the batch?	Yes — degrades with very small batches	No — each sample is masked independently
At inference	Uses stored running mean and variance	Disabled entirely; full network used
Effect on training speed	Substantially **faster** convergence	**Slower** convergence (typically 2–3× more epochs)
Effect on learning rate	Allows much larger η	No direct effect

Aspect	Batch Normalization	Dropout
Typical placement	After the linear layer, before the activation	After the activation, in fully connected layers
Regularization strength	Mild (a side effect)	Strong (its main purpose)

7. Using them together

BN and dropout are complementary — BN mainly helps *optimisation* while dropout mainly helps *generalisation* — and they are often combined. However, they interact awkwardly: dropout changes the variance of the activations between training and test time, while BN's running statistics were estimated under the training-time (dropped) distribution, producing a **variance shift** that can hurt accuracy. The practical conventions that emerged are:

- Place dropout **after** the BN layer, never between the linear layer and BN.
- In modern convolutional architectures (ResNet, EfficientNet), BN alone is often sufficient and dropout is used only in the final classifier layer.
- In Transformers, **Layer Normalization + dropout** is the standard pairing, with dropout applied to attention weights and residual branches.
- When both are used, reduce the dropout rate (e.g. 0.1–0.2 rather than 0.5), since BN already contributes noise.

Question 7 | Overfitting, Underfitting and the Bias–Variance Trade-off

> *Explain overfitting and underfitting in deep neural networks. Discuss the bias–variance trade-off.*

1. The goal: generalization

The objective of training is **not** to minimise training error — that is merely the quantity we can measure. The real objective is to minimise error on data the model has never seen. The gap between these two is the whole subject of this question. Overfitting and underfitting are the two ways a model can fail to generalise, and they fail in opposite directions.

2. Underfitting (high bias)

Underfitting occurs when the model is too simple or too constrained to capture the underlying structure of the data. It performs poorly on the training set *and* on the validation set, and the two errors are similar and both high.

- **Causes:** too few layers or neurons; excessive regularization (λ too large, dropout rate too high); training stopped too early; learning rate badly chosen; poor or insufficient input features.
- **Signature:** the training loss plateaus at a high value and refuses to fall further, and the training and validation curves sit close together.
- **Remedies:** increase capacity (more layers, more units), train for longer, reduce regularization, engineer better features, or use a more suitable architecture.

3. Overfitting (high variance)

Overfitting occurs when the model has enough capacity to memorise the training data, including its noise and irrelevant idiosyncrasies. It performs excellently on the training set but poorly on unseen data.

- **Causes:** model capacity too large relative to the amount of data; too few training samples; training for too many epochs; noisy or mislabelled data; features that leak information specific to the training set.
- **Signature:** training loss keeps falling towards zero while validation loss reaches a minimum and then **starts to rise** — the widening gap between the two curves is the definitive diagnostic.
- **Remedies:** add regularization (L2, dropout), gather more data, apply data augmentation, use early stopping, reduce capacity, or add batch normalisation.

4. The bias–variance decomposition

For squared-error loss, the expected error of a model on an unseen point decomposes exactly into three parts:

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Var}[\hat{f}(x)] + \sigma^2_{\text{noise}}$$

Term	Meaning	Behaviour
Bias²	The error from wrong assumptions — how far the *average* model (over many training sets) is from the true function.	**Decreases** as model capacity increases. High bias = underfitting.
Variance	The error from sensitivity to the particular training sample — how much the learned model changes if you retrain on a different sample of the same size.	**Increases** as model capacity increases. High variance = overfitting.
Irreducible noise σ²	Inherent randomness in the data itself (measurement error, genuine ambiguity, label noise).	**Cannot be reduced by any model.** It sets the floor on achievable error.

5. The trade-off, visualised

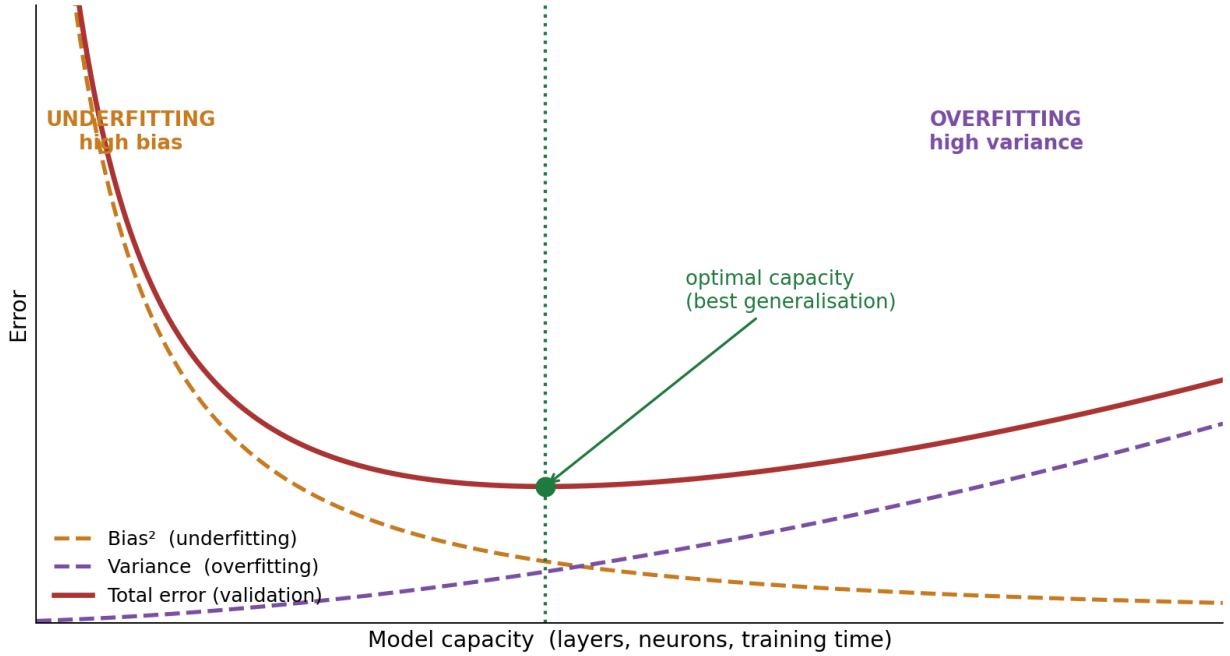


Figure 7.1 — Bias² falls and variance rises with model capacity; total error is minimised at an intermediate point.

The **trade-off** is that bias and variance cannot generally be reduced simultaneously by adjusting capacity: reducing one increases the other. The total error curve is therefore U-shaped, and the goal is to locate its minimum — the capacity at which the sum of the two is smallest. This is the point of best generalisation, and it is found empirically using a validation set.

6. Comparison summary

Aspect	Underfitting (high bias)	Good fit	Overfitting (high variance)
Training error	High	Low	Very low / near zero
Validation error	High	Low	High
Gap between them	Small	Small	**Large and growing**
What was learned	Not even the underlying pattern	The underlying pattern	The pattern **plus** the noise
Decision boundary	Too smooth / too simple	Appropriately shaped	Excessively convoluted
Fix	More capacity, train longer, less regularization	Keep it	More data, augmentation, regularization, early stopping

7. Diagnosing which problem you have

1. Plot **training loss and validation loss on the same axes against epoch**. This single plot answers the question.
2. Both high and close together → **underfitting**. Increase capacity or train longer before doing anything else.
3. Training low, validation high, gap growing → **overfitting**. Add regularization or data.
4. Both low and close → a good fit. Stop and evaluate once on the test set.

5. If validation loss is *lower* than training loss, this is usually not a paradox — dropout and other noise are active during training but not during evaluation.

8. A modern caveat — double descent

The classical U-shaped curve is not the complete story for very large neural networks. Empirically, as capacity is increased past the point where the model can interpolate the training data exactly, test error often **falls again** — a phenomenon known as **double descent**. This is why enormous over-parameterised models can generalise well despite classical theory predicting catastrophic overfitting, and it is an active area of research. The classical trade-off remains the correct guiding framework for the moderate-sized networks encountered in most practical work.

Question 8 | Comparison of Gradient Descent, Momentum, RMSProp and Adam

> *Compare standard Gradient Descent, Momentum-based Gradient Descent, RMSProp, and Adam with respect to gradient handling and convergence behaviour.*

1. Update rules side by side

Standard GD :

$$w_t = w_{t-1} - \eta \cdot g_t$$

Momentum :

$$v_t = \beta \cdot v_{t-1} + \eta \cdot g_t$$
$$w_t = w_{t-1} - v_t$$

RMSProp :

$$s_t = \beta \cdot s_{t-1} + (1 - \beta) \cdot g_t^2$$
$$w_t = w_{t-1} - \eta \cdot g_t \,/\, (\sqrt{s_t} + \varepsilon)$$

Adam :

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$
$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$
$$\hat{m}_t = m_t / (1 - \beta_1^{\,t}), \quad \hat{v}_t = v_t / (1 - \beta_2^{\,t})$$
$$w_t = w_{t-1} - \alpha \cdot \hat{m}_t \,/\, (\sqrt{\hat{v}_t} + \varepsilon)$$

2. How each handles the gradient

Optimizer	What it does with the gradient	State stored per parameter
Standard GD	Uses the raw current gradient directly. No memory of past gradients; the same η is applied to every parameter.	None
Momentum	Accumulates an exponentially weighted average of past gradients into a velocity. Consistent directions reinforce; oscillating components cancel. Uses the **first moment** (direction).	1 value (velocity v)
RMSProp	Divides the gradient by the RMS of its own recent magnitudes, so each parameter gets an individually scaled step. Uses the **second moment** (magnitude). Direction is unmodified.	1 value (squared-gradient average s)
Adam	Combines both: momentum for direction and RMS scaling for magnitude, plus bias correction for the zero initialisation.	2 values (m and v)

3. Convergence behaviour

Optimizer	Path through the loss surface	Convergence characteristics
Standard GD	Follows the steepest descent direction exactly. In a narrow ravine it zig-zags violently across the walls while creeping slowly along the floor.	Slow but predictable and well understood. Highly sensitive to η. Stalls at saddle points and plateaus where $g \approx 0$. Guaranteed to converge on convex problems for suitable η.
Momentum	Behaves like a heavy ball rolling downhill. Oscillations perpendicular to the valley cancel out; motion along the valley accumulates and accelerates.	Substantially faster than plain GD on ill-conditioned surfaces. Powers through small local minima, plateaus and saddle points. Can **overshoot** the minimum and require several steps to settle back.
RMSProp	Takes steps of roughly uniform size in every direction, because each is normalised by its own	Handles badly scaled and non-stationary objectives well. Effective for RNNs. Lacking

Optimizer	Path through the loss surface	Convergence characteristics
	gradient scale. Steep directions are damped, flat directions amplified.	momentum, it can still be noisy and may oscillate near the minimum.
Adam	Smooth, well-damped and consistently scaled — momentum gives a stable direction while RMS scaling gives a stable magnitude.	Fastest and most reliable initial convergence in most settings. Very robust to the choice of α. Occasionally converges to slightly worse final minima than well-tuned SGD+momentum on large vision benchmarks.

4. Practical comparison

Criterion	GD	Momentum	RMSProp	Adam
Adaptive per-parameter step	No	No	**Yes**	**Yes**
Uses momentum	No	**Yes**	No	**Yes**
Bias correction	—	No	No	**Yes**
Memory overhead per parameter	0	1×	1×	2×
Hyperparameters	η	η, β	η, β, ε	α, β_1, β_2, ε
Typical default	$\eta = 0.01$–0.1	$\eta = 0.01$, $\beta = 0.9$	$\eta = 0.001$, $\beta = 0.9$	$\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$
Sensitivity to η	Very high	High	Moderate	**Low**
Escapes saddle points	Poorly	Well	Moderately	**Well**
Handles sparse gradients	Poorly	Poorly	Well	**Very well**
Best used for	Teaching, convex problems	Vision tasks with careful tuning	RNNs, non-stationary objectives	**General default**, Transformers, GANs

5. Summary of the progression

The four methods form a natural sequence in which each addition fixes a specific defect of its predecessor:

- **GD → Momentum** fixes *oscillation and slow progress in ravines* by adding memory of the gradient direction.
- **GD → RMSProp** fixes *badly scaled gradients across parameters* by adding memory of gradient magnitude.
- **Momentum + RMSProp → Adam** fixes both at once, and the bias correction additionally fixes the *underestimated first few steps* that arise from initialising the moving averages at zero.

In current practice, **Adam and AdamW are the default choice** for most new work, while **SGD with momentum** together with a good learning-rate schedule remains competitive — and sometimes marginally superior in final test accuracy — on well-studied large-scale vision problems.

Question 9 | Numerical — Parameter Count of a 5–4–3–2 Feedforward Network

> *A feedforward neural network consists of an input layer with 5 neurons, Hidden Layer 1 with 4 neurons, Hidden Layer 2 with 3 neurons, and an output layer with 2 neurons. All consecutive layers are fully connected and each neuron except input neurons has one bias. (i) Illustrate the network architecture. (ii) Evaluate the number of weights between each pair of layers. (iii) Evaluate the total number of trainable parameters including biases.*

Given data

```
Input layer     : 5 neurons
Hidden layer 1  : 4 neurons  (each with 1 bias)
Hidden layer 2  : 3 neurons  (each with 1 bias)
Output layer    : 2 neurons  (each with 1 bias)
Connectivity    : fully connected between consecutive layers
```

(i) Network architecture

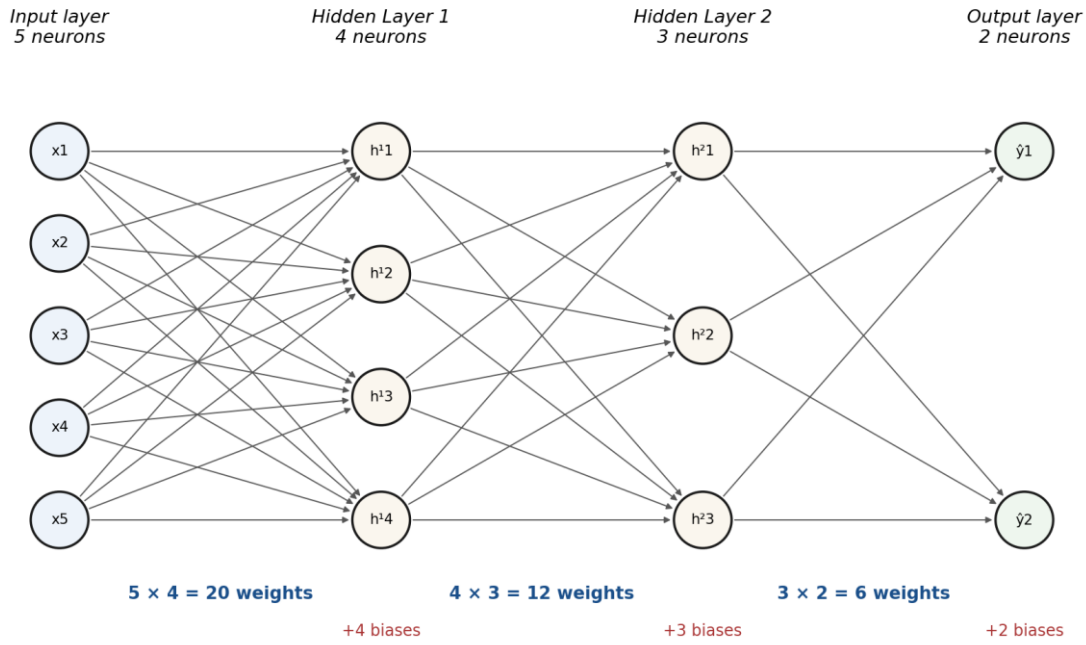


Figure 9.1 — Fully connected 5–4–3–2 feedforward network with all inter-layer connections shown.

Note that the **input layer carries no biases and performs no computation** — it merely distributes the feature values. Biases exist only on the 4 + 3 + 2 = 9 computational neurons.

(ii) Weights between each pair of layers

For a fully connected pair of layers, the number of weights is (neurons in the previous layer) × (neurons in the current layer), since every neuron in one layer connects to every neuron in the next.

```
Input → Hidden 1  : 5 × 4 = 20 weights
Hidden 1 → Hidden 2 : 4 × 3 = 12 weights
Hidden 2 → Output : 3 × 2 =  6 weights
                    ____________________
```

Total weights = 20 + 12 + 6 = 38

(iii) Total trainable parameters

Step 1 — count the biases. One bias per non-input neuron:

```
Hidden layer 1 : 4 biases
Hidden layer 2 : 3 biases
Output layer   : 2 biases
                 _________

Total biases   : 9
```

Step 2 — add weights and biases:

```
Total trainable parameters = total weights + total biases
                           = 38 + 9
Total trainable parameters = 47
```

Layer-wise breakdown

Connection / Layer	Weight calculation	Weights	Biases	Parameters
Input → Hidden 1	5 × 4	20	4	24
Hidden 1 → Hidden 2	4 × 3	12	3	15
Hidden 2 → Output	3 × 2	6	2	8
Total	—	**38**	**9**	**47**

General formula and verification

For a network with layer sizes n_0, n_1, …, n_L (where n_0 is the input layer), the total parameter count is:

$$P = \sum_{l=1}^{L} (n_{l-1} \cdot n_l + n_l) = \sum_{l=1}^{L} n_l \cdot (n_{l-1} + 1)$$

Applying this to our network:

```
P = 4(5 + 1) + 3(4 + 1) + 2(3 + 1)
  = 4(6) + 3(5) + 2(4)
  = 24 + 15 + 8
P = 47   ✓  (confirms the layer-wise result)
```

Observations

- **The bias term is equivalent to an extra input of constant value 1.** This is why the formula can be written as n_l·(n_{l−1} + 1) — the '+1' is the bias, treated as one more incoming connection.
- **Weights dominate the count.** 38 of the 47 parameters (81%) are weights. In real networks the imbalance is far more extreme: a layer of 1024 → 1024 units has 1,048,576 weights but only 1,024 biases, so biases account for under 0.1% of parameters.
- **The first layer holds the most parameters here** (24 of 47), because the input layer is the widest. In practice this is why fully connected layers applied directly to images are impractical — a 224×224×3 image has 150,528 inputs, so a single 1000-unit dense layer would need over 150 million weights. This is the motivation for convolutional layers with shared weights.
- **Effect of widening a layer.** If hidden layer 1 grew from 4 to 6 neurons, the count would become 6(6) + 3(7) + 2(4) = 36 + 21 + 8 = **65** — an increase of 18 parameters for just two extra neurons, because the change affects the layers on *both* sides.
- **Parameters versus data.** A useful sanity check is that the number of training samples should comfortably exceed the number of parameters; where it does not, regularization becomes essential (Question 1).

Question 10 | Numerical — Two RMSProp Update Steps

> *For RMSProp, $v_t = \beta v_{(t-1)} + (1-\beta) g_t^2$ and $w_t = w_{(t-1)} - \eta g_t / (\sqrt{v_t} + \varepsilon)$. Given $w_0 = 2.0$, $v_0 = 0$, $\beta = 0.9$, $\eta = 0.01$, $\varepsilon \approx 0$, and gradients $g_1 = 0.8$ and $g_2 = 0.4$, calculate: (i) v_1 and w_1, (ii) v_2 and w_2.*

Given data

Initial parameter : $w_0 = 2.0$

Initial accumulator : $v_0 = 0$

Decay rate : $\beta = 0.9$ Learning rate : $\eta = 0.01$

Epsilon : $\varepsilon \approx 0$ Gradients : $g_1 = 0.8$, $g_2 = 0.4$

$$v_t = \beta \cdot v_{(t-1)} + (1-\beta) \cdot g_t^2$$

$$w_t = w_{(t-1)} - \eta \cdot g_t / (\sqrt{v_t} + \varepsilon)$$

(i) First iteration — v_1 and w_1

Step 1 — update the squared-gradient accumulator:

$$v_1 = \beta \cdot v_0 + (1-\beta) \cdot g_1^2$$
$$= (0.9)(0) + (0.1)(0.8)^2$$
$$= 0 + (0.1)(0.64)$$
$$\mathbf{v_1 = 0.064}$$

Step 2 — compute the root:

$$\sqrt{v_1} = \sqrt{0.064} = 0.252982$$

Step 3 — update the parameter:

$$w_1 = w_0 - \eta \cdot g_1 / \sqrt{v_1}$$
$$= 2.0 - (0.01)(0.8) / 0.252982$$
$$= 2.0 - 0.008 / 0.252982$$
$$= 2.0 - 0.031623$$
$$\mathbf{w_1 = 1.968377 \quad (step\ size = 0.031623)}$$

(ii) Second iteration — v_2 and w_2

Step 1 — update the accumulator (now $v_1 \neq 0$, so both terms contribute):

$$v_2 = \beta \cdot v_1 + (1-\beta) \cdot g_2^2$$
$$= (0.9)(0.064) + (0.1)(0.4)^2$$
$$= 0.0576 + (0.1)(0.16)$$
$$= 0.0576 + 0.016$$
$$\mathbf{v_2 = 0.0736}$$

Step 2 — compute the root:

$$\sqrt{v_2} = \sqrt{0.0736} = 0.271293$$

Step 3 — update the parameter:

$$w_2 = w_1 - \eta \cdot g_2 / \sqrt{v_2}$$
$$= 1.968377 - (0.01)(0.4) / 0.271293$$
$$= 1.968377 - 0.004 / 0.271293$$
$$= 1.968377 - 0.014744$$
$$\mathbf{w_2 = 1.953633 \quad (step\ size = 0.014744)}$$

Summary table

t	g_t	g_t^2	v_t	$\sqrt{v_t}$	Step $\eta \cdot g_t / \sqrt{v_t}$	w_t
0	—	—	0	—	—	2.000000
1	0.8	0.64	**0.064000**	0.252982	0.031623	**1.968377**
2	0.4	0.16	**0.073600**	0.271293	0.014744	**1.953633**

Interpretation and key observations

- **The step is not simply η·g.** With plain gradient descent the first step would be (0.01)(0.8) = 0.008. RMSProp instead took a step of 0.031623 — about **four times larger** — because dividing by √v = 0.253 amplified it. RMSProp rescales rather than merely scaling the gradient.
- **The gradient halved but the step did not.** From t = 1 to t = 2 the gradient fell from 0.8 to 0.4 (a factor of 2), yet the step fell from 0.0316 to 0.0147 (a factor of 2.15). The denominator changed only slightly, because the moving average with β = 0.9 has memory of the earlier large gradient. This *smoothing* is exactly the intended behaviour: RMSProp responds to sustained trends rather than to individual noisy gradients.
- **The normalising effect.** If the gradient were perfectly constant at value g, then in steady state $v \to g^2$ and $\sqrt{v} \to |g|$, so the step would approach $\eta \cdot g/|g| = \eta$ regardless of the gradient's magnitude. RMSProp therefore tends towards steps of roughly constant size η — which is precisely how it equalises progress across parameters with wildly different gradient scales.
- **Effect of the initialisation $v_0 = 0$.** Because $v_1 = 0.1 \cdot g_1^2$ rather than g_1^2, the first denominator $\sqrt{0.064} = 0.253$ is much smaller than $|g_1| = 0.8$, inflating the very first step. This is the **initialisation bias** that plain RMSProp does not correct; Adam adds an explicit bias-correction term to remove it (Question 11).
- **Role of ε.** Here $\varepsilon \approx 0$ was assumed, but in a real implementation $\varepsilon \approx 10^{-8}$ guards against division by zero when a parameter's gradients have been zero for a long stretch.
- **Direction of movement.** Both gradients are positive, so both updates decrease w — the parameter is moving consistently downhill from 2.0 towards 1.954.

Question 11 | Numerical — One Complete Adam Update with Bias Correction

> *A neural network parameter is updated using Adam. Given $w_0 = 1.50$, $g_1 = 0.40$, $m_0 = 0$, $v_0 = 0$, $\alpha = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$, evaluate: (i) m_1, (ii) v_1, (iii) the bias-corrected estimates $\hat{m}_1$ and $\hat{v}_1$, and (iv) the updated parameter w_1. Use $m_t = \beta_1 m_{(t-1)} + (1 - \beta_1) g_t$ and $v_t = \beta_2 v_{(t-1)} + (1 - \beta_2) g_t^2$.*

Given data

$w_0 = 1.50 \qquad g_1 = 0.40$

$m_0 = 0 \qquad v_0 = 0$

$\alpha = 0.01 \qquad \beta_1 = 0.9 \qquad \beta_2 = 0.999 \qquad \varepsilon = 10^{-8}$

$$m_t = \beta_1 \cdot m_{(t-1)} + (1 - \beta_1) \cdot g_t \qquad \text{(first moment)}$$

$$v_t = \beta_2 \cdot v_{(t-1)} + (1 - \beta_2) \cdot g_t^2 \qquad \text{(second moment)}$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t) \qquad \text{(bias correction)}$$

$$w_t = w_{(t-1)} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

(i) First moment estimate m_1

$$m_1 = \beta_1 \cdot m_0 + (1 - \beta_1) \cdot g_1$$
$$= (0.9)(0) + (1 - 0.9)(0.40)$$
$$= 0 + (0.1)(0.40)$$
$$\mathbf{m_1 = 0.04}$$

(ii) Second moment estimate v_1

$$v_1 = \beta_2 \cdot v_0 + (1 - \beta_2) \cdot g_1^2$$
$$= (0.999)(0) + (1 - 0.999)(0.40)^2$$
$$= 0 + (0.001)(0.16)$$
$$\mathbf{v_1 = 0.00016}$$

(iii) Bias-corrected estimates

At t = 1, the correction denominators are:

$$1 - \beta_1^t = 1 - 0.9^1 = 1 - 0.9 = 0.1$$
$$1 - \beta_2^t = 1 - 0.999^1 = 1 - 0.999 = 0.001$$

Corrected first moment:

$$\hat{m}_1 = m_1 / (1 - \beta_1^1)$$
$$= 0.04 / 0.1$$
$$\mathbf{\hat{m}_1 = 0.4}$$

Corrected second moment:

$$\hat{v}_1 = v_1 / (1 - \beta_2^1)$$
$$= 0.00016 / 0.001$$
$$\mathbf{\hat{v}_1 = 0.16}$$

Notice that $\hat{m}_1 = 0.4 = g_1$ and $\hat{v}_1 = 0.16 = g_1^2$. With only one observation available, the corrected estimates recover the observation exactly — which is precisely what an unbiased estimator should do, and confirms the correction is working.

(iv) Updated parameter w_1

$$\sqrt{\hat{v}_1} = \sqrt{0.16} = 0.4$$

$$w_1 = w_0 - \alpha \cdot \hat{m}_1 / (\sqrt{\hat{v}_1} + \varepsilon)$$
$$= 1.50 - (0.01)(0.4) / (0.4 + 10^{-8})$$
$$= 1.50 - 0.004 / 0.4$$
$$= 1.50 - 0.01$$
$$\mathbf{w_1 = 1.49 \quad (step\ magnitude = 0.01 = \alpha)}$$

Summary table

Quantity	Formula	Substitution	Result
m_1	$\beta_1 m_0 + (1 - \beta_1) g_1$	0.9(0) + 0.1(0.40)	**0.04**
v_1	$\beta_2 v_0 + (1 - \beta_2) g_1^2$	0.999(0) + 0.001(0.16)	**0.00016**
$\hat{m}_1$	$m_1/(1 - \beta_1^1)$	0.04 / 0.1	**0.4**
$\hat{v}_1$	$v_1/(1 - \beta_2^1)$	0.00016 / 0.001	**0.16**
$\sqrt{\hat{v}_1}$	$\sqrt{0.16}$	—	0.4
w_1	$w_0 - \alpha \cdot \hat{m}_1/(\sqrt{\hat{v}_1} + \varepsilon)$	1.50 − 0.01(0.4)/0.4	**1.49**

Why bias correction matters — a direct comparison

Suppose the correction had been omitted and the raw m_1 and v_1 were used instead:

$$\text{Uncorrected : } w_1 = 1.50 - (0.01)(0.04)/(\sqrt{0.00016})$$
$$= 1.50 - 0.0004 / 0.012649$$
$$= 1.50 - 0.031623$$
$$= 1.468377$$

The uncorrected step of 0.0316 is more than **three times larger** than the correct step of 0.01 — and in the wrong way: because m and v are initialised to zero, m is under-estimated by a factor of 10 while v is under-estimated by a factor of 1000, and $\sqrt{1000} \approx 31.6$ outweighs the factor of 10. The result is a wildly inflated first step, exactly the instability that RMSProp suffers from and Adam is designed to avoid.

Further observations

- **The step size equals α exactly.** Since $\hat{m}_1 = g_1$ and $\sqrt{\hat{v}_1} = |g_1|$, the ratio $\hat{m}_1/\sqrt{\hat{v}_1} = 1$ and the update is exactly $\alpha = 0.01$. This illustrates Adam's most useful property: the step magnitude is **bounded by roughly α, independent of the gradient's scale**. A gradient of 0.4, 400 or 0.0004 would all have produced a first step of 0.01.
- **This is why Adam's default α = 0.001 works so widely** — it directly sets the maximum distance a parameter can move per step, in parameter units, regardless of how the loss happens to be scaled.
- **The correction factors decay quickly.** By t = 50, $1 - \beta_1^t = 0.9948$ and by t = 1000, $1 - \beta_2^t = 0.632$. Bias correction matters most in the first tens (for m) to thousands (for v) of iterations, and becomes negligible thereafter.
- **ε is negligible here.** With $\sqrt{\hat{v}_1} = 0.4$, adding 10^{-8} changes nothing; ε only matters when $\hat{v}$ is extremely small.
- **Direction.** $g_1 > 0$, so the parameter decreases from 1.50 to 1.49 — correctly moving down the slope.

Question 12 | Numerical — Adam Step Magnitude and Direction Reversal

> *A parameter is optimized using Adam. At a particular iteration, $\hat{m}_t = 0.24$ and $\hat{v}_t = 0.09$. Current parameter $w_t = 2.50$. Use $\alpha = 0.02$ and $\varepsilon \approx 0$. Calculate: (i) parameter update magnitude, (ii) updated parameter value. Then, if gradient direction reverses so that $\hat{m}_t = -0.18$ while $\hat{v}_t = 0.09$, calculate the new update and comment on the direction of parameter change.*

Given data

Current parameter : $w_t = 2.50$
Corrected first moment : $\hat{m}_t = 0.24$
Corrected second moment : $\hat{v}_t = 0.09$
Step size $\alpha = 0.02$, $\varepsilon \approx 0$

Update rule : $\Delta w = \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$
$w_{new} = w_t - \Delta w$

(i) Parameter update magnitude

Step 1 — take the square root of the second moment:

$\sqrt{\hat{v}_t} = \sqrt{0.09} = 0.3$

Step 2 — compute the update:

$$\Delta w = \alpha \cdot \hat{m}_t / \sqrt{\hat{v}_t}$$
$$= (0.02)(0.24) / 0.3$$
$$= 0.0048 / 0.3$$
$$\mathbf{\Delta w = 0.016 \quad (update\ magnitude = 0.016)}$$

Equivalently, the *normalised gradient* is $\hat{m}/\sqrt{\hat{v}} = 0.24/0.3 = 0.8$, and the update is $\alpha \times 0.8 = 0.02 \times 0.8 = 0.016$.

(ii) Updated parameter value

$$w_{new} = w_t - \Delta w$$
$$= 2.50 - 0.016$$
$$\mathbf{w_{new} = 2.484}$$

Since $\hat{m}_t$ is **positive**, the accumulated gradient points in the positive direction, so gradient descent moves the parameter in the **negative** direction — the parameter decreases from 2.50 to 2.484.

(iii) After the gradient direction reverses

Now $\hat{m}_t = -0.18$ with $\hat{v}_t$ unchanged at 0.09. Note that $\hat{v}$ is an average of **squared** gradients, so it is always non-negative and is unaffected by a sign change — only $\hat{m}$ carries directional information.

$\sqrt{\hat{v}_t} = \sqrt{0.09} = 0.3$ (unchanged)

$$\Delta w = \alpha \cdot \hat{m}_t / \sqrt{\hat{v}_t}$$
$$= (0.02)(-0.18) / 0.3$$
$$= -0.0036 / 0.3$$
$$\mathbf{\Delta w = -0.012 \quad (magnitude\ 0.012,\ negative\ sign)}$$

New parameter value:

$$w_{new} = 2.484 - (-0.012)$$
$$= 2.484 + 0.012$$

```
w_new = 2.496
```

Summary table

Case	$\hat{m}_t$	$\sqrt{\hat{v}_t}$	$\hat{m}_t/\sqrt{\hat{v}_t}$	$\Delta w = \alpha \cdot \hat{m}_t/\sqrt{\hat{v}_t}$	w before	w after	Direction
Original	+0.24	0.3	+0.80	**+0.016**	2.500	**2.484**	Decreases ↓
Reversed	−0.18	0.3	−0.60	**−0.012**	2.484	**2.496**	Increases ↑

Comment on the direction of parameter change

- **The direction of movement reverses.** In the first case Δw is positive, so w = 2.50 − 0.016 decreases. In the second case Δw is negative, so subtracting it *adds* to w and the parameter increases from 2.484 to 2.496. The parameter has turned around and is now heading back the way it came.
- **The sign is carried entirely by $\hat{m}_t$.** Because $\sqrt{\hat{v}_t}$, α and ε are all strictly positive, sign(Δw) = sign($\hat{m}_t$). The second moment controls only *how far* the step goes, never *which way*.
- **Why $\hat{v}$ does not change sign.** $\hat{v}$ is built from g^2, so a gradient of −0.18 and one of +0.18 contribute identically. This is by design: $\hat{v}$ is meant to measure the *typical magnitude* of the gradient, which is a scale, not a direction.
- **The reversed step is smaller** (0.012 versus 0.016) purely because $|\hat{m}|$ is smaller (0.18 versus 0.24) while the denominator is unchanged. The ratio of the steps, 0.012/0.016 = 0.75, exactly equals 0.18/0.24 ✓.
- **What a sign reversal means physically.** The optimiser has stepped past a minimum: the slope now points the other way, so the correction pulls it back. Overshoot followed by correction is normal and is how gradient methods settle into a minimum. Momentum makes this damping smooth — because $\hat{m}$ is an exponentially weighted average, one contrary gradient does not immediately flip the direction; the average must genuinely change sign first, which filters out noise.
- **A caution.** If the sign of $\hat{m}$ alternates on every iteration with a large magnitude, the optimiser is oscillating rather than converging, and α should be reduced or a decay schedule applied.

ADDITIONAL QUESTIONS (13 – 17)

Question 13 | Learning-Rate Schedules and Hyperparameter Tuning

> *Explain why a fixed learning rate is rarely optimal throughout training. Describe the common learning-rate schedules and explain the main strategies for hyperparameter tuning in deep networks.*

1. Why a fixed learning rate is rarely optimal

Training has two distinct phases with conflicting requirements. **Early in training** the parameters are far from any good solution, the loss surface is being crossed at a coarse scale, and a large learning rate is desirable to make rapid progress and to escape poor regions. **Late in training** the optimiser is already inside a good basin and needs to settle precisely into its floor; a large step now simply bounces around the minimum and prevents convergence. A single fixed η must compromise between these, and the compromise is usually poor at both ends.

The standard solution is a **learning-rate schedule**: start large and decrease η over time according to a predetermined rule. Empirically, the characteristic loss curve of a scheduled run shows a sharp additional drop each time the rate is reduced — evidence that the optimiser was oscillating within a basin it could not previously enter.

2. The common schedules

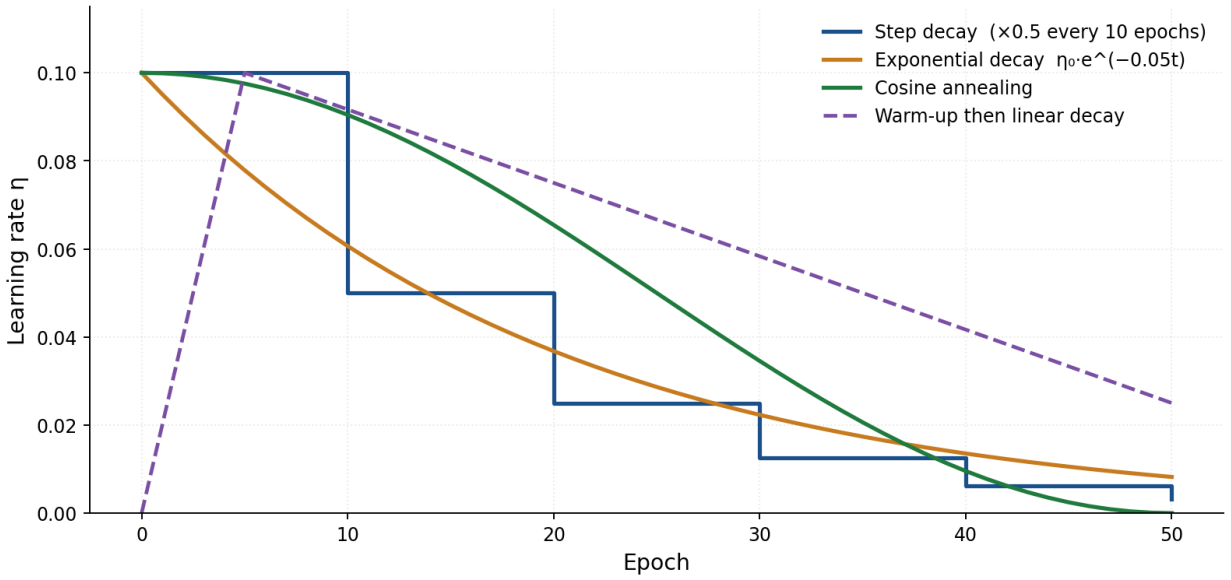


Figure 13.1 — Four widely used learning-rate schedules over 50 epochs, all starting from $\eta_0 = 0.1$.

Schedule	Formula	Characteristics
Step decay	$\eta = \eta_0 \cdot \gamma^{\lfloor t/k \rfloor}$	Multiply by a factor γ (typically 0.1 or 0.5) every k epochs. Simple, interpretable, and historically the standard for ImageNet training. The abrupt drops are visible as sudden falls in the loss curve.
Exponential decay	$\eta = \eta_0 \cdot e^{(-kt)}$	Smooth continuous decay. Easy to reason about, but requires tuning the decay constant k, and can decay too aggressively early on.

Schedule	Formula	Characteristics
Cosine annealing	$\eta = \frac{1}{2}\eta_0[1 + \cos(\pi t/T)]$	Decays slowly at first, quickly in the middle, then slowly again near zero. Now the most popular choice for modern architectures — it spends useful time at both high and low rates.
Warm-up	η rises linearly from ~0 to η_0 over the first few epochs	Used before any of the above. Early gradients are large and the moment estimates unreliable, so a full-size step can destabilise the model. Essential for Transformers and very large batch sizes.
Warm restarts (SGDR)	Cosine annealing repeated in cycles, with η jumping back up at the start of each cycle	The periodic jumps let the optimiser escape a basin and explore others, and the collection of end-of-cycle models can be used as a free ensemble.
ReduceLROnPlateau	Reduce η by a factor when the validation metric stops improving for a set patience	Adaptive rather than pre-scheduled — reacts to what the model is actually doing instead of following a fixed clock.

3. Worked example — step decay

With $\eta_0 = 0.1$, $\gamma = 0.5$ and $k = 10$ epochs, the schedule $\eta = \eta_0 \cdot 0.5^{\lfloor t/10 \rfloor}$ gives:

```
Epoch  0 : η = 0.1 × 0.5⁰ = 0.100000
Epoch 10 : η = 0.1 × 0.5¹ = 0.050000
Epoch 20 : η = 0.1 × 0.5² = 0.025000
Epoch 30 : η = 0.1 × 0.5³ = 0.012500
Epoch 40 : η = 0.1 × 0.5⁴ = 0.006250
```

For comparison, cosine annealing over $T = 50$ epochs gives $\eta = \frac{1}{2}(0.1)[1 + \cos(\pi \cdot 25/50)] =$ **0.05** at the halfway point — the same value the step schedule reaches at epoch 10, illustrating how much more gradually cosine annealing decays in its early phase.

4. The main hyperparameters to tune

Hyperparameter	Typical range	Priority
Learning rate η / α	$10^{-5} – 10^{-1}$ (log scale)	**Highest** — tune this first, always
Batch size	16 – 512 (powers of 2)	High — interacts strongly with η
Number of layers / units	problem-dependent	High — sets model capacity
Weight decay λ	$10^{-6} – 10^{-2}$ (log scale)	Medium
Dropout rate p	0.1 – 0.5	Medium
Optimizer choice	Adam, AdamW, SGD+momentum	Medium
Momentum β / β_1, β_2	0.9, 0.999 (defaults rarely need changing)	Low
ε	10^{-8}	Very low — essentially never tuned

5. Tuning strategies

- **Grid search** — evaluate every combination on a predefined grid. Exhaustive and reproducible, but the cost grows exponentially with the number of hyperparameters, and it wastes effort on dimensions that turn out not to matter.

- **Random search** — sample combinations at random from specified distributions. Bergstra and Bengio (2012) showed this is *more efficient than grid search* for the same budget, because in practice only a few hyperparameters matter and random sampling explores many distinct values of each, whereas a grid repeatedly retests the same values of the important one.
- **Bayesian optimisation** — build a probabilistic surrogate model of the validation score as a function of the hyperparameters and use it to choose the most informative next trial. Far more sample-efficient than random search, at the cost of extra complexity.
- **Successive halving / Hyperband** — start many configurations with a small budget, discard the worst performers, and give the survivors more budget. Extremely effective when a configuration's early performance predicts its final performance.
- **Coarse-to-fine manual search** — begin with a wide log-scale sweep of η, identify the promising decade, then refine within it. This remains the most common approach in practice and is often the fastest for a small number of hyperparameters.

6. Practical rules

- **Sample learning rate and weight decay on a logarithmic scale**, not a linear one. The difference between 0.001 and 0.01 matters enormously; the difference between 0.091 and 0.1 does not.
- **Tune on a validation set, never on the test set.** The test set is used exactly once, at the end, to report an unbiased estimate.
- **Change one thing at a time** when tuning manually, and log every run with its full configuration and result.
- **η and batch size are coupled.** A common heuristic is the *linear scaling rule*: if the batch size is multiplied by k, multiply η by k as well (with warm-up to keep the first steps stable).
- **Use a fixed random seed** during tuning so that differences between runs reflect the hyperparameters rather than initialisation noise — then verify the winner across several seeds.

Question 14 | Numerical — Plain Gradient Descent versus Momentum

> *For the objective $J(w) = w^2$ with gradient $dJ/dw = 2w$, starting from $w_0 = 1.0$ with $\eta = 0.1$, perform four iterations of (a) plain gradient descent and (b) gradient descent with momentum using $\beta = 0.9$ and the update $v_t = \beta v_{t-1} + \eta \cdot g_t$, $w_t = w_{t-1} - v_t$. Compare the two trajectories and comment on the behaviour of momentum.*

Given data

Objective : $J(w) = w^2$ Gradient : $dJ/dw = 2w$

Start : $w_0 = 1.0$ Minimum at $w^* = 0$

Learning rate : $\eta = 0.1$ Momentum : $\beta = 0.9$, $v_0 = 0$

Plain GD : $w_t = w_{(t-1)} - \eta \cdot g_t$

Momentum : $v_t = \beta \cdot v_{(t-1)} + \eta \cdot g_t$; $w_t = w_{(t-1)} - v_t$

(a) Plain gradient descent

Iteration 1 : $g_1 = 2(1.000) = 2.000$

$w_1 = 1.000 - (0.1)(2.000) = 0.800$

Iteration 2 : $g_2 = 2(0.800) = 1.600$

$w_2 = 0.800 - (0.1)(1.600) = 0.640$

Iteration 3 : $g_3 = 2(0.640) = 1.280$

$w_3 = 0.640 - (0.1)(1.280) = 0.512$

Iteration 4 : $g_4 = 2(0.512) = 1.024$

$w_4 = 0.512 - (0.1)(1.024) = 0.4096$

Plain GD after 4 steps : $w_4 = 0.4096$, $J = 0.167772$

(b) Gradient descent with momentum

Iteration 1 : $g_1 = 2(1.000) = 2.000$

$v_1 = (0.9)(0) + (0.1)(2.000) = 0.200$

$w_1 = 1.000 - 0.200 = 0.800$

Iteration 2 : $g_2 = 2(0.800) = 1.600$

$v_2 = (0.9)(0.200) + (0.1)(1.600)$

$= 0.180 + 0.160 = 0.340$

$w_2 = 0.800 - 0.340 = 0.460$

Iteration 3 : $g_3 = 2(0.460) = 0.920$

$v_3 = (0.9)(0.340) + (0.1)(0.920)$

$= 0.306 + 0.092 = 0.398$

$w_3 = 0.460 - 0.398 = 0.062$

Iteration 4 : $g_4 = 2(0.062) = 0.124$

$v_4 = (0.9)(0.398) + (0.1)(0.124)$

$= 0.3582 + 0.0124 = 0.3706$

$w_4 = 0.062 - 0.3706 = -0.3086$

Momentum after 4 steps : $w_4 = -0.3086$, $J = 0.095234$

Side-by-side comparison

Iteration	Plain GD: w_t	Plain GD: $J(w_t)$	Momentum: v_t	Momentum: w_t	Momentum: $J(w_t)$
0 (start)	1.0000	1.000000	0.0000	1.0000	1.000000
1	0.8000	0.640000	0.2000	0.8000	0.640000
2	0.6400	0.409600	0.3400	**0.4600**	0.211600
3	0.5120	0.262144	0.3980	**0.0620**	**0.003844**
4	0.4096	0.167772	0.3706	**−0.3086**	0.095234

Commentary on the results

- **The first step is identical** (both reach 0.8000), because $v_o = 0$ means there is no accumulated velocity yet to distinguish the two methods.
- **Momentum accelerates rapidly thereafter.** By iteration 3 momentum has reached w = 0.0620 — essentially at the optimum, with J = 0.0038 — while plain GD is still at 0.5120 with J = 0.2621. Momentum's loss is roughly **68 times smaller** at that point. This is the acceleration effect: because every gradient points in the same direction, the velocity keeps growing (0.200 → 0.340 → 0.398) and each step is larger than the raw gradient step would be.
- **Momentum overshoots at iteration 4.** The accumulated velocity of 0.3706 carries w past zero to −0.3086. This is the characteristic cost of momentum: it behaves like a heavy ball, and a heavy ball does not stop instantly at the bottom of a valley.
- **The overshoot is self-correcting.** At w = −0.3086 the gradient becomes negative, so the velocity is progressively reduced and then reversed, and w oscillates back with decreasing amplitude until it settles at zero. What momentum trades is a small amount of oscillation near the optimum for a large amount of speed getting there.
- **In one dimension the overshoot looks like a defect; in high dimensions it is a net win.** Real loss surfaces are ill-conditioned ravines. Momentum cancels the oscillating components across the narrow direction (where consecutive gradients alternate in sign) while accumulating the consistent component along the valley floor. Both effects come from the same averaging.
- **Effective step amplification.** In the limit where the gradient is constant, the velocity converges to $v = \eta \cdot g/(1 - \beta) = \eta \cdot g/0.1 =$ **$10 \cdot \eta \cdot g$** for $\beta = 0.9$. Momentum can therefore take steps up to ten times larger than plain GD, which is exactly the factor by which it speeds up progress along a consistent direction.
- **Practical implication.** $\beta = 0.9$ is the standard default. Larger β (0.95–0.99) gives more acceleration but more overshoot, and usually requires a correspondingly smaller η. **Nesterov momentum** refines the method by evaluating the gradient at the *look-ahead* position $w - \beta v$, which anticipates the overshoot and damps it.

Question 15 | Early Stopping, Data Augmentation and Cross-Validation

> *Explain early stopping, data augmentation and cross-validation. Discuss how each contributes to reliable generalization, and how the training, validation and test sets should be used.*

1. Why these three belong together

Questions 1 and 6 covered regularization techniques that act on the *model* — penalising weights or perturbing activations. The three techniques in this question act on the *training process* and the *data* instead. They cost nothing in terms of model capacity, are architecture-independent, and in practice are often more effective than any weight penalty. All three depend on a correct data split, which is where we begin.

2. The three-way data split

Set	Typical share	Purpose	How often it is used
Training set	60–80%	The data the optimiser actually fits — gradients are computed from it.	Every iteration
Validation set	10–20%	Model selection: choosing hyperparameters, deciding when to stop, comparing architectures.	Once per epoch
Test set	10–20%	A single unbiased estimate of generalisation performance on truly unseen data.	**Exactly once**, at the very end

The cardinal rule is that **the test set must never influence any decision**. Each time a choice is made on the basis of test performance, information leaks from the test set into the model, and the reported accuracy becomes optimistically biased. Repeatedly tuning against the test set is one of the most common and most damaging methodological errors in applied machine learning.

3. Early stopping

Early stopping monitors the validation loss at the end of every epoch and halts training once it stops improving. Concretely:

1. Track the best validation loss seen so far and the epoch at which it occurred.
2. After each epoch, if the validation loss improves, save a checkpoint of the model.
3. If it fails to improve for a fixed number of consecutive epochs (the **patience**, typically 5–20), stop training.
4. Restore the checkpoint from the best epoch, not the final one.

- **Why it works.** During training, a model first learns the broad structure of the data (which reduces both training and validation error) and only later begins to memorise noise (which reduces training error while *increasing* validation error). Early stopping halts training precisely at the transition.
- **It is a form of regularization.** Limiting the number of optimisation steps limits how far the weights can travel from their small initial values, which is closely related to an L2 penalty — for a quadratic loss the correspondence is exact.
- **Patience matters.** Validation loss is noisy, so a patience of 1 stops far too early on a random fluctuation. Patience must be large enough to see past normal epoch-to-epoch variation.
- **It is essentially free.** No extra parameters, no extra computation, and it also saves training time. There is rarely a reason not to use it.

4. Data augmentation

Data augmentation artificially enlarges the training set by applying transformations that change the input while **preserving the label**. Because the model never sees exactly the same example twice, memorisation becomes far harder, and the model is explicitly taught which variations are irrelevant.

Domain	Common augmentations
Images	Horizontal flips, random crops and resizing, small rotations, brightness/contrast/saturation jitter, random erasing (Cutout), Mixup (blending two images and their labels), CutMix
Text	Synonym replacement, random word deletion or swap, back-translation (translate to another language and back), sentence shuffling
Audio	Time shifting, pitch shifting, time stretching, adding background noise, SpecAugment (masking bands of the spectrogram)
Tabular	SMOTE for minority-class oversampling, adding small Gaussian noise to continuous features

- **It encodes invariances.** Horizontally flipping a photograph of a cat still shows a cat, so flipping teaches the model that left–right orientation is irrelevant. This is genuine prior knowledge being injected into training.
- **The transformation must be label-preserving.** Flipping a photograph of the digit '2' horizontally does *not* produce a valid '2', and vertical flips are wrong for most natural photographs. Choosing augmentations requires domain understanding.
- **It is often the single most effective anti-overfitting measure**, frequently outperforming any amount of weight decay or dropout — because the underlying problem is usually insufficient data, and augmentation attacks that directly.
- **Apply it to the training set only.** Augmenting the validation or test set corrupts the evaluation. (Test-time augmentation, where predictions are averaged over several augmented copies of a test input, is a separate deliberate technique.)

5. Cross-validation

When the dataset is small, a single train/validation split is unreliable: the estimate depends heavily on which samples happened to land in the validation set. **k-fold cross-validation** removes this dependence:

1. Split the data into k equally sized folds (k = 5 or 10 are standard).
2. For each fold i: train on the other k − 1 folds and evaluate on fold i.
3. Average the k scores to obtain the cross-validated estimate, and report the standard deviation as a measure of its reliability.

- **Every sample is used for both training and validation**, just never at the same time, which makes efficient use of scarce data.
- **The standard deviation across folds is itself informative** — a large spread indicates that the model is unstable or the dataset too small to draw firm conclusions.
- **Stratified k-fold** preserves the class proportions within each fold and should always be used for classification with imbalanced classes.
- **Cost.** It requires k full training runs, which is why it is standard in classical machine learning but comparatively rare in deep learning, where a single run can take days. Deep learning normally relies on a single large held-out validation set instead.

- **Respect the data's structure.** For time series use forward-chaining splits, never random folds, since training on the future to predict the past leaks information. Similarly, group k-fold must be used when multiple samples come from the same patient, speaker or subject.

6. How they combine in practice

1. Split the data into train / validation / test, stratifying and respecting group or temporal structure.
2. Apply data augmentation to the **training set only**.
3. Train with early stopping monitored on the validation set, saving the best checkpoint.
4. Use the validation set to select hyperparameters (learning rate, weight decay, dropout, architecture).
5. Where data is scarce, replace the single validation split with k-fold cross-validation for hyperparameter selection.
6. Retrain the final chosen configuration and evaluate **once** on the test set to report performance.

Question 16 | Numerical — L1 versus L2 Penalty Gradients and Inverted Dropout Scaling

> *(a) A weight currently has value w = 0.8 and a data-loss gradient ∂L_data/∂w = 0.1. Using λ = 0.01 and η = 0.1, compute one update step with no regularization, with L2 regularization, and with L1 regularization. (b) Repeat the penalty-gradient comparison for a small weight w = 0.01 and explain why L1 produces sparsity while L2 does not. (c) A hidden layer outputs h = [2.0, 4.0, 6.0, 8.0]. Inverted dropout is applied with drop probability p = 0.5 and the sampled keep-mask is [1, 0, 1, 0]. Compute the layer output during training and at test time.*

PART (a) — One update step under each regime

```
Given : w = 0.8,  ∂L_data/∂w = 0.1,  λ = 0.01,  η = 0.1

No regularization : ∂J/∂w = ∂L_data/∂w
L2 (J = L + λw²)  : ∂J/∂w = ∂L_data/∂w + 2λw
L1 (J = L + λ|w|) : ∂J/∂w = ∂L_data/∂w + λ·sign(w)
```

No regularization

```
∂J/∂w = 0.1
w_new = 0.8 − (0.1)(0.1) = 0.8 − 0.01
```

w_new = 0.79

With L2 regularization

```
Penalty gradient = 2λw = 2(0.01)(0.8) = 0.016
∂J/∂w = 0.1 + 0.016 = 0.116
w_new = 0.8 − (0.1)(0.116) = 0.8 − 0.0116
```

w_new = 0.7884

With L1 regularization

```
Penalty gradient = λ·sign(0.8) = (0.01)(+1) = 0.01
∂J/∂w = 0.1 + 0.01 = 0.11
w_new = 0.8 − (0.1)(0.11) = 0.8 − 0.011
```

w_new = 0.789

Comparison at w = 0.8

Regime	Penalty gradient	Total gradient	Step η·g	w_new	Extra shrinkage
No regularization	0	0.1000	0.01000	**0.790000**	—
L2 (λ = 0.01)	2λw = 0.016	0.1160	0.01160	**0.788400**	0.001600
L1 (λ = 0.01)	λ·sign(w) = 0.010	0.1100	0.01100	**0.789000**	0.001000

PART (b) — Behaviour at a small weight, and why L1 gives sparsity

Now take w = 0.01 and recompute only the penalty gradients:

Weight w	L2 penalty gradient (2λw)	L1 penalty gradient (λ·sign w)	Ratio L1 : L2
0.80	2(0.01)(0.80) = **0.016000**	0.01 × (+1) = **0.010000**	0.63 : 1
0.01	2(0.01)(0.01) = **0.000200**	0.01 × (+1) = **0.010000**	**50 : 1**
0.001	2(0.01)(0.001) = **0.000020**	0.01 × (+1) = **0.010000**	**500 : 1**

This table is the whole explanation of sparsity. The L2 penalty gradient is *proportional to w*, so as a weight shrinks the force pushing it towards zero shrinks with it — at w = 0.001 the pull is a negligible 0.00002, and the weight simply lingers near zero without ever reaching it. The L1 penalty gradient is **constant at λ regardless of how small w is**, so a weight of 0.001 is pushed towards zero exactly as hard as a weight of 0.8.

Concretely, suppose the data gradient for some weight has become negligible (the feature is irrelevant). Under L1 the weight then evolves as:

```
w = 0.010 → 0.010 − (0.1)(0.01) = 0.009
w = 0.009 → 0.008 → 0.007 → … → 0.001 → 0
```

It marches to zero in equal decrements and then stays there, because the sub-gradient convention sign(0) = 0 holds it fixed. (Practical implementations use a proximal or *soft-thresholding* step that clips the weight to exactly zero when the update would carry it across.) Under L2 the same weight would decay geometrically — 0.010, 0.00998, 0.00996 … — approaching zero asymptotically but never arriving.

L1 → weights hit exactly zero → sparse model, feature selection
L2 → weights shrink but stay non-zero → dense model, small smooth weights

PART (c) — Inverted dropout

```
Given : h = [2.0, 4.0, 6.0, 8.0]
        drop probability p = 0.5,  keep probability q = 1 − p = 0.5
        sampled keep-mask r = [1, 0, 1, 0]
        h̃ = (r ⊙ h) / q
```

Step 1 — apply the mask (element-wise multiplication):

```
r ⊙ h = [1×2.0, 0×4.0, 1×6.0, 0×8.0]
      = [2.0, 0.0, 6.0, 0.0]
```

Step 2 — rescale by 1/q = 1/0.5 = 2:

```
h̃ = [2.0×2, 0.0×2, 6.0×2, 0.0×2]
```

Training output : h̃ = [4.0, 0.0, 12.0, 0.0]

At test time, dropout is switched off entirely and no scaling is applied:

Test output : h̃ = [2.0, 4.0, 6.0, 8.0] (unchanged)

Why the 1/q rescaling is necessary

Consider any single unit j. During training its output is h_j/q with probability q and 0 with probability 1 − q, so its **expected** output is:

$$E[\tilde{h}_j] = q \cdot (h_j/q) + (1 - q) \cdot 0 = h_j$$

The expected activation during training therefore equals the deterministic activation used at test time. Without the 1/q factor, the expectation would be $q \cdot h_j = 0.5h_j$, so every downstream neuron would receive inputs of roughly half the magnitude at training time compared with test time — a systematic mismatch that would corrupt the predictions.

- **Note that a single sampled mask will not match the expectation.** Here the four training outputs average 4.0 while the test outputs average 5.0. The guarantee is about the *expected value per unit* over many masks, not about any one draw. The variance introduced by that mismatch is precisely the noise that makes dropout regularise.

- **Why 'inverted'.** The original formulation scaled by q at *test* time instead. The inverted version is preferred because it puts all the extra arithmetic in the training path and leaves inference as a plain, unmodified forward pass — simpler, faster, and impossible to get wrong when deploying.
- **Verification of the mask.** With p = 0.5 and four units, the expected number kept is 2, which matches the sampled mask here. A different sampled mask would be drawn on the very next training step, and over an epoch the network sees many different thinned sub-networks.

Question 17 | The Loss Landscape — Local Minima, Saddle Points and Ill-Conditioning

> *Describe the structure of the loss landscape of a deep neural network. Explain local minima, saddle points, plateaus and ill-conditioning, and discuss why gradient-based optimization succeeds in practice despite the problem being non-convex.*

1. The nature of the problem

Training a deep network means minimising J(θ), a scalar function of millions of parameters. Unlike linear or logistic regression, this function is **non-convex**: it has many critical points, and there is no guarantee that gradient descent will find the global minimum. Classical optimisation theory therefore predicts that training should be extremely difficult — yet in practice it works reliably. Understanding why requires understanding what the landscape actually looks like in very high dimensions.

2. Features of the landscape

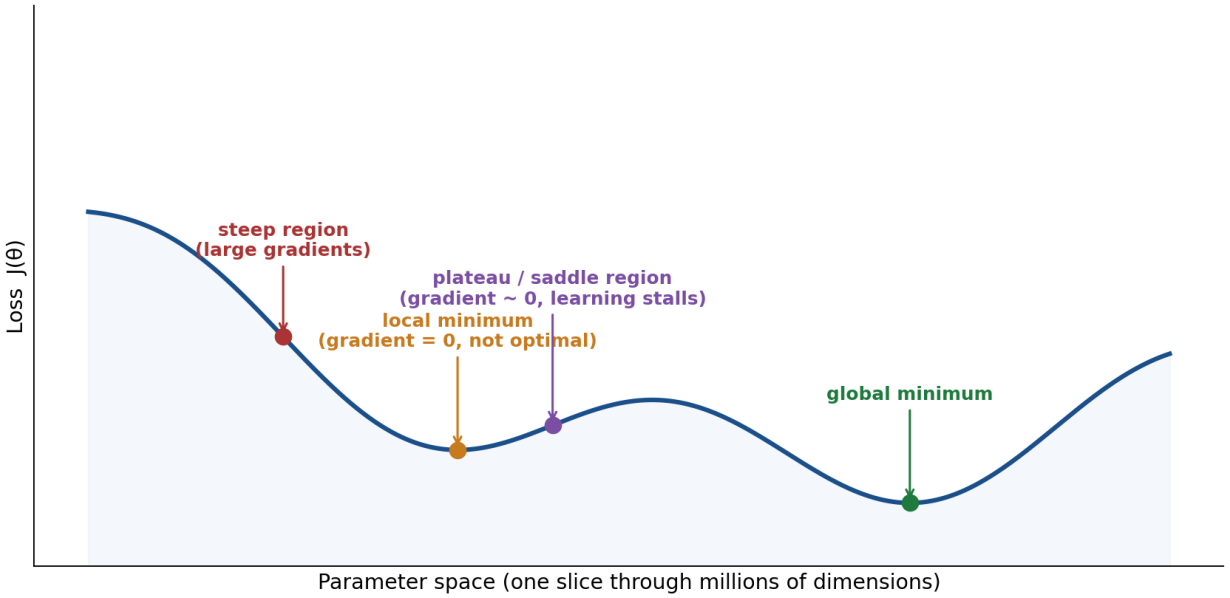


Figure 17.1 — A one-dimensional slice through a loss landscape, showing the features that affect optimisation.

Feature	Definition	Effect on training
Global minimum	The point of lowest loss over the entire parameter space.	The theoretical target. In practice it is neither found nor needed — many distinct minima give equally good test performance.
Local minimum	A point where the gradient is zero and the loss increases in *every* direction (the Hessian is positive definite).	Gradient descent halts here. Classically feared, but in high dimensions genuine local minima are rare and are usually close in value to the global minimum.
Saddle point	A point where the gradient is zero but the loss increases in some directions and decreases in others (the Hessian has both positive and negative eigenvalues).	**The dominant obstacle in deep learning.** Gradient descent slows dramatically near a saddle because g ≈ 0, even though an escape direction exists.
Plateau	A large flat region where the gradient is very small but not exactly zero.	Training appears to stall for many epochs before suddenly resuming. Often caused by saturated activations.

Feature	Definition	Effect on training
Ravine / ill-conditioning	A region where curvature is far greater in some directions than others (the Hessian has a large condition number).	Plain gradient descent zig-zags across the steep walls while creeping along the valley floor — the classic motivation for momentum and adaptive methods.
Cliff	A region of extremely steep gradient, common in recurrent networks.	A single step can jump an enormous distance and destroy the model. This is what gradient clipping exists to prevent.

3. Why saddle points, not local minima, are the real problem

At a critical point the Hessian has n eigenvalues, one per parameter. The point is a local minimum only if **all n** are positive; if even one is negative, it is a saddle. Under a rough random-matrix argument in which each eigenvalue is positive with probability about ½, the chance of a critical point being a genuine local minimum is roughly 2^{-n}. For n in the millions this is vanishingly small: **almost every critical point in a deep network is a saddle point.**

Dauphin et al. (2014) confirmed this empirically and also found a systematic structure: critical points with high loss are overwhelmingly saddles, while the rare local minima cluster at low loss values close to the global minimum. This explains the central empirical observation of deep learning — *which* minimum is reached matters much less than commonly assumed, because most of them are about equally good.

4. Why gradient-based optimization succeeds anyway

- **Most minima are good.** Because low-loss regions are broad and interconnected, the practical question is not 'did we find the global minimum' but 'did we reach the low-loss region', and the answer is usually yes.
- **Stochastic noise escapes saddles.** A mini-batch gradient is not the true gradient; its noise perturbs the parameters off the saddle's ridge, providing exactly the nudge needed to find a descent direction. This is one of the strongest arguments for SGD over full-batch descent.
- **Momentum carries the optimiser through flat regions.** Accumulated velocity does not vanish just because the current gradient does, so the optimiser coasts across plateaus and saddles instead of stopping on them.
- **Adaptive methods inflate steps where gradients are small.** RMSProp and Adam divide by √v; in a flat region v is small, so the effective step grows and progress continues.
- **Over-parameterisation smooths the landscape.** Wider networks have more paths to a low-loss solution, and empirically their loss surfaces are markedly less rugged. This is a strong practical reason to start with a network larger than strictly necessary and then regularise it.
- **Skip connections flatten the landscape.** Visualisations by Li et al. (2018) showed that adding residual connections transforms a chaotic, highly non-convex surface into a much smoother, near-convex bowl — a large part of why very deep ResNets train at all.
- **Normalisation improves conditioning.** Batch normalisation reduces the Hessian's condition number, directly attacking the ravine problem and permitting much larger learning rates.

5. Flat versus sharp minima

Not all minima with the same training loss generalise equally well. A **flat minimum** sits in a broad basin where the loss changes slowly with the parameters; a **sharp minimum** sits in a narrow spike. Because the test loss

surface is a slightly shifted version of the training one, a small shift barely changes the loss at a flat minimum but can change it enormously at a sharp one — so **flat minima generalise better.**

- Small batch sizes produce noisier gradients, which cannot settle into narrow spikes and therefore tend to find flatter minima. This partly explains why very large batch training often generalises worse unless the learning rate is scaled up and warm-up is used.
- Dropout, weight decay and data augmentation all bias training towards solutions that remain good under perturbation — that is, towards flat minima.
- Sharpness-Aware Minimisation (SAM) makes this explicit by optimising the worst-case loss within a small neighbourhood of the current parameters.

6. Practical implications

1. Do not attempt to find the global minimum; aim for a low-loss region that generalises. Validation performance, not training loss, is the criterion.
2. Use stochastic mini-batches rather than full-batch descent — the noise is beneficial, not merely a computational compromise.
3. Use momentum or an adaptive optimiser so that plateaus and saddles do not stall training.
4. Use skip connections and normalisation in deep architectures to keep the landscape well conditioned.
5. Apply gradient clipping where cliffs are likely, particularly in recurrent networks.
6. If training stalls, suspect a plateau or saddle before assuming the model has converged — try a learning-rate warm restart before abandoning the run.